

Graphs

What is a Graph

A graph is a non-linear data structure made of nodes (vertices) connected by edges. Unlike trees, graphs can have cycles and don't need a single root. Graphs model real relationships: social networks, maps, dependency chains, and networks.

Types of Graphs

- Directed vs Undirected: edges have a direction (one-way) or not (two-way).
- Weighted vs Unweighted: edges carry a cost/distance value or not.
- Cyclic vs Acyclic: contains at least one cycle, or none (a DAG - Directed Acyclic Graph - is common in scheduling problems).
- Connected vs Disconnected: every vertex is reachable from every other, or not.

Representations

- Adjacency Matrix: a 2D array where $\text{matrix}[i][j] = 1$ (or weight) if an edge exists between i and j . $O(V^2)$ space, $O(1)$ edge lookup.
- Adjacency List: each vertex stores a list of its neighbors. $O(V+E)$ space, more efficient for sparse graphs — this is the most common representation in interviews.

Traversal Algorithms

- BFS (Breadth-First Search): explores neighbor by neighbor using a queue. Finds the shortest path in an unweighted graph.
- DFS (Depth-First Search): explores as far as possible down one path before backtracking, using recursion or a stack. Used for cycle detection, topological sort, connected components.

BFS and DFS in Python

```
from collections import deque, defaultdict
```

```
graph = defaultdict(list)
```

```
graph[0] = [1, 2]
```

```
graph[1] = [2]
```

```
graph[2] = [0, 3]
```

```
def bfs(start):
```

```
    visited = set([start])
```

```

queue = deque([start])

order = []

while queue:
    node = queue.popleft()
    order.append(node)
    for neighbor in graph[node]:
        if neighbor not in visited:
            visited.add(neighbor)
            queue.append(neighbor)

return order

```

```

def dfs(node, visited=None, order=None):
    if visited is None:
        visited, order = set(), []
    visited.add(node)
    order.append(node)
    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(neighbor, visited, order)

    return order

```

Important Algorithms

- Dijkstra's Algorithm: shortest path from one source to all vertices in a weighted graph with non-negative weights, $O((V+E) \log V)$ with a min-heap.
- Topological Sort: linear ordering of vertices in a DAG such that every edge goes from earlier to later in the order — used in task scheduling, build systems.
- Union-Find (Disjoint Set): efficiently tracks connected components, key for Kruskal's Minimum Spanning Tree algorithm and cycle detection.
- Kruskal's / Prim's Algorithm: find a Minimum Spanning Tree (cheapest way to connect all vertices).

Common Interview Problems

- Number of Islands (grid-based DFS/BFS).
- Detect a cycle in a directed or undirected graph.

- Clone a graph.
- Course Schedule — detect if all courses can be finished (cycle detection via topological sort).
- Shortest path in a weighted graph (Dijkstra).
- Number of connected components in an undirected graph.